



# Software Factory en la Era de la IA

Estructura

|               |                      |
|---------------|----------------------|
| Organización  | lean-ai              |
| Clasificación | Generado desde sf-kb |
| Fecha         | 13 de julio de 2026  |
| Versión       | 1.0                  |

## Organizacion Tipica

# Organización Típica de una Software Factory

Cómo se estructura una Software Factory a nivel organizacional: jerarquía, departamentos, tamaño y líneas de reporte.

## Modelo Jerárquico Estándar

La mayoría de Software Factories siguen un patrón jerárquico que equilibra 交付能力 con agilidad organizacional.



## Departamentos Típicos

| Departamento      | Responsabilidad principal                                | Tamaño típico (SF mediana) |
|-------------------|----------------------------------------------------------|----------------------------|
| Engineering       | Desarrollo de software, code review, technical decisions | 60-70% del personal        |
| Quality Assurance | Testing, automation, quality gates                       | 10-15%                     |
| DevOps / SRE      | CI/CD, infraestructura, observabilidad                   | 5-10%                      |
| Product           | Backlog, roadmap, user research                          | 5-8%                       |
| Design            | UX/UI, design system, research                           | 3-5%                       |
| Architecture      | Standards, ADRs, tech radar                              | 3-5%                       |
| PMO               | Gestión de proyectos, reporting, process improvement     | 3-5%                       |

## Líneas de Reporte

### Modelo funcional (tradicional)

- Reporte vertical por especialidad
- Los devs reportan a un Engineering Manager
- QA reporta a un QA Manager
- **Ventaja:** profundidad técnica, carrera clara
- **Desventaja:** silos, comunicación cruzada lenta

## Modelo por producto

- Equipos multidisciplinarios dedicados a un producto
- Cada equipo tiene dev, QA, design, PO
- **Ventaja:** ownership, velocidad de decisión
- **Desventaja:** duplicación de roles especializados

## Modelo matricial (híbrido)

- Reporte dual: funcional + por proyecto/producto
- El individual contributor tiene dos jefes
- **Ventaja:** flexibilidad de recursos
- **Desventaja:** conflicto de prioridades, complejidad

Ver Estructura Matricial para análisis detallado.

## Tamaños Típicos por Etapa

| Etapa      | Total empleados | Ingenieros | Ratio eng:soporte |
|------------|-----------------|------------|-------------------|
| Startup SF | 10-30           | 8-25       | 4:1               |
| SF mediana | 50-200          | 35-140     | 3.5:1             |
| SF grande  | 200-500         | 140-350    | 3:1               |
| Enterprise | 500+            | 350+       | 2.5:1             |

A medida que la SF crece, la proporción de roles de soporte (management, HR, operations) aumenta.

## Conway y la Organización

La Ley de Conway establece que los sistemas reflejan la estructura organizacional. Esto implica:

- Si tienes silos, el software tendrá acoplamiento entre módulos
- Si tienes equipos autónomos, tenderás a microservicios
- Los boundaries de equipo deben mapear a los boundaries del sistema

## Patrones Organizacionales Comunes

| Patrón       | Cuándo usarlo                    | Ejemplo                                   |
|--------------|----------------------------------|-------------------------------------------|
| Feature Team | Producto con features frecuentes | Equipo de pagos, equipo de notificaciones |

| Patrón                | Cuándo usarlo                       | Ejemplo                               |
|-----------------------|-------------------------------------|---------------------------------------|
| Platform Team         | Múltiples equipos consumiendo infra | Equipo de platform engineering        |
| Enabling Team         | Transiciones técnicas temporales    | Migración a Kubernetes                |
| Complicated Subsystem | Componente con alta complejidad     | Motor de ML, motor de reglas fiscales |

Ver Team Topologies para el framework completo.

## Métricas de Salud Organizacional

| Métrica                  | Rango saludable | Señal de alerta |
|--------------------------|-----------------|-----------------|
| Lead time                | < 5 días        | > 15 días       |
| Deploy frequency         | > 1/semana      | < 1/mes         |
| Team satisfaction (eNPS) | > 30            | < 0             |
| Bus factor por squad     | >= 3            | = 1             |
| Time to onboard          | < 30 días       | > 90 días       |

Ver Métricas para mediciones detalladas.

## Anti-patrones Organizacionales

1. **Conway invertido**: forzar la organización a copiar la arquitectura actual
2. **Hero culture**: dependencia de individuos clave en lugar de procesos
3. **Committee-driven architecture**: decisiones técnicas por consenso burocrático
4. **Resource pooling**: asignar personas a múltiples proyectos simultáneamente
5. **Zero support staff**: ingenieros haciendo tareas de ops, admin, o soporte

## Referencias

- ¿Qué es una Software Factory?
- Producto vs Proyecto
- Gobernanza Técnica
- Cultura de Equipo

## Roles Equipo

# Roles del Equipo de Desarrollo

Mapa completo de roles en una Software Factory moderna: responsabilidades, skills requeridas y cómo interactúan entre sí.

## Roles de Ingeniería

### Software Developer / Engineer

| Aspecto           | Detalle                                           |
|-------------------|---------------------------------------------------|
| Responsabilidad   | Escribir, mantener y revisar código de producción |
| Skills clave      | Algoritmos, data structures, frameworks, testing  |
| Niveles           | Junior → Mid → Senior → Staff → Principal         |
| Entrega principal | Feature code, bug fixes, technical documentation  |

### QA Engineer / SDET

| Aspecto           | Detalle                                             |
|-------------------|-----------------------------------------------------|
| Responsabilidad   | Garantizar calidad del software antes de producción |
| Skills clave      | Testing strategies, automation, performance testing |
| Niveles           | QA → Senior QA → QA Lead → QA Architect             |
| Entrega principal | Test plans, automated suites, quality reports       |

### DevOps Engineer

| Aspecto           | Detalle                                           |
|-------------------|---------------------------------------------------|
| Responsabilidad   | CI/CD, infraestructura, deployment automation     |
| Skills clave      | Containers, IaC, CI/CD pipelines, cloud platforms |
| Niveles           | DevOps → Senior DevOps → DevOps Lead              |
| Entrega principal | Pipelines, infrastructure as code, runbooks       |

### Site Reliability Engineer (SRE)

| Aspecto           | Detalle                                             |
|-------------------|-----------------------------------------------------|
| Responsabilidad   | Reliability, observabilidad, incident management    |
| Skills clave      | SLOs/SLIs, distributed systems, monitoring, on-call |
| Niveles           | SRE → Senior SRE → SRE Manager                      |
| Entrega principal | SLOs, incident reports, reliability improvements    |

## Platform Engineer

| Aspecto           | Detalle                                                 |
|-------------------|---------------------------------------------------------|
| Responsabilidad   | Internal Developer Platform, golden paths, self-service |
| Skills clave      | Kubernetes, service mesh, developer experience          |
| Niveles           | Platform Eng → Senior → Platform Lead                   |
| Entrega principal | IDP, templates, documentation, tooling                  |

## Software Architect

Ver Software Architect para detalles completos.

## Tech Lead

Ver Tech Lead para detalles completos.

## Roles de Producto

### Product Owner / Business Analyst

Ver Product Owner para detalles completos.

## UX/UI Designer

| Aspecto           | Detalle                                                    |
|-------------------|------------------------------------------------------------|
| Responsabilidad   | Research, wireframes, prototyping, design system           |
| Skills clave      | User research, Figma, accessibility, design thinking       |
| Niveles           | Designer → Senior Designer → Design Lead → Design Director |
| Entrega principal | Mockups, prototypes, design system components              |

## Scrum Master / Agile Coach

| Aspecto           | Detalle                                           |
|-------------------|---------------------------------------------------|
| Responsabilidad   | Facilitar procesos ágiles, remover impedimentos   |
| Skills clave      | Facilitation, coaching, conflict resolution       |
| Entrega principal | Sprint health, team metrics, process improvements |

## Roles de Management

### Engineering Manager

Ver Engineering Manager para detalles completos.

### Project Manager / Delivery Manager

| Aspecto           | Detalle                                                      |
|-------------------|--------------------------------------------------------------|
| Responsabilidad   | Timeline, budget, stakeholder communication, risk management |
| Skills clave      | Planning, communication, risk assessment, reporting          |
| Entrega principal | Project plans, status reports, risk registers                |

## Matriz de Interacción entre Roles



## Distribución Típica por Equipo (Squad de 8-12)

| Rol                 | Cantidad                 | % del squad |
|---------------------|--------------------------|-------------|
| Software Developers | 4-6                      | 50%         |
| QA / SDET           | 1-2                      | 15%         |
| DevOps / Platform   | 1                        | 10%         |
| Product Owner       | 1                        | 10%         |
| UX Designer         | 1                        | 10%         |
| Scrum Master        | 1 (puede ser compartido) | 5%          |

En equipos más pequeños, los roles se combinan: un senior developer puede actuar como tech lead, un QA

puede cubrir DevOps tasks.

## Evolución de Roles con IA

La IA está transformando los roles tradicionales:

| Rol       | Impacto de IA                                        |
|-----------|------------------------------------------------------|
| Developer | Copilot + code generation, más focus en architecture |
| QA        | AI-assisted testing, predictive quality              |
| DevOps    | AIOps, self-healing infrastructure                   |
| Architect | AI-powered design analysis, ADR generation           |
| PM/PO     | Automated reporting, data-driven prioritization      |

## Anti-patronos de Roles

1. **Senior everywhere**: equipo de puros seniors sin mentorship pipeline
2. **Role silos**: "eso no es mi trabajo" culture
3. **Missing QA**: asumir que developers testing = quality
4. **Architect as decision maker**: architect decide todo sin input del team
5. **SM as project manager**: Scrum Master enfocado en tracking, no en coaching

## Referencias

- Organización Típica
- Team Topologies
- El Manifiesto Ágil
- Cultura de Equipo

---

## Tech Lead

# Tech Lead: Rol y Responsabilidades

El Tech Lead es el puente entre la visión técnica y la ejecución del equipo. Combina coding hands-on con liderazgo técnico.

## ¿Qué es un Tech Lead?

Un Tech Lead es un ingeniero senior que asume responsabilidades de liderazgo técnico **sin dejar de escribir código**. No es un rol de management — es un rol técnico con influencia organizacional.

"A Tech Lead is a software engineer who is responsible for the technical quality of the team's output." — Will Larson, *An Elegant Puzzle*

## Responsabilidades Core

### Decisiones Técnicas

- Evaluar y seleccionar tecnologías, frameworks, herramientas
- Definir approach técnico para features complejos
- Balancear entre "perfecto" y "good enough" (time-to-market)
- Tomar decisiones de ADR cuando corresponda

### Code Review

- Establecer estándares de code review en el equipo
- Revisar PRs críticos, particularmente los de arquitectura
- Enforce coding standards y patterns
- Mentorear a developers junior en reviews

### Architecture Guidance

- Diseñar la solución técnica a nivel de feature/sprint
- Identificar y mitigar technical debt
- Definir interfaces entre componentes
- Coordinar con el Software Architect para decisions de system-wide

### Mentoring y Coaching

- Guiar el crecimiento técnico del equipo
- Pair programming sessions
- Tech talks internos
- Code review como herramienta de enseñanza

## Communication Bridge

- Traducir business requirements a technical specifications
- Comunicar constraints y tradeoffs al Product Owner
- Escalar blockers al Engineering Manager

## Tech Lead vs Otros Roles

| Aspecto             | Tech Lead                  | Software Architect     | Engineering Manager            |
|---------------------|----------------------------|------------------------|--------------------------------|
| Escribe código      | Sí (50-70%)                | Opcionalmente (20-40%) | Raramente (<10%)               |
| Decisiones técnicas | Feature-level              | System-wide            | Delega al Tech Lead            |
| People management   | No directo                 | No                     | Sí (1:1s, hiring, performance) |
| Scope               | 1 squad / team             | Múltiples sistemas     | Múltiples personas             |
| Reporting           | EM o Arquitecto            | VP Engineering         | Director / VP                  |
| Accountability      | Technical quality del team | Architecture quality   | People & delivery              |

## Días Típicos de un Tech Lead

| Hora  | Actividad                                 |
|-------|-------------------------------------------|
| 09:00 | Sprint standup, revisar blockers          |
| 09:30 | Code review block (2-3 PRs)               |
| 10:30 | Pair programming con mid-level dev        |
| 11:30 | Tech design session para feature upcoming |
| 12:00 | Lunch                                     |
| 13:00 | 1:1 con EM o Arquitecto                   |
| 13:30 | Architecture discussion con architect     |
| 14:00 | Coding block (feature implementation)     |
| 16:00 | Cross-team sync (dependencias)            |
| 16:30 | Sprint planning / refinement prep         |
| 17:00 | Documentation, ADR writing                |

## Habilidades Clave

### Técnicas

- Profundidad en el tech stack del equipo

- Conocimiento de Arquitectura Limpia y design patterns
- Comprensión de tradeoffs (CAP theorem, performance vs maintainability)
- Testing strategies y engineering practices

## Soft Skills

- **Influencia sin autoridad:** motivar al equipo sin ser su jefe directo
- **Comunicación:** explicar decisiones técnicas a audiencias no-técnicas
- **Priorización:** balancear tech debt, features, y bugs
- **Conflict resolution:** mediar desacuerdos técnicos

## Errores Comunes

| Error                   | Consecuencia                 | Alternativa                        |
|-------------------------|------------------------------|------------------------------------|
| Código en solitario     | Bottleneck, knowledge silo   | Pair programming, distribute       |
| No delegar              | Burnout, team growth stunted | Mentorear, trust the team          |
| Ignorar tech debt       | Velocity decay               | Dedicate 20% a refactor            |
| Micro-managing          | Team autonomy erosion        | Define standards, let team execute |
| Solo coding, no leading | Team lacks direction         | Balance coding & leadership        |

## Tech Lead como IC Track

En muchas Software Factories, el Tech Lead es el primer paso del **Individual Contributor (IC)** track:

IC Track: IC → Senior IC → Staff Engineer → Principal → Distinguished  
 ‡ (puede transicionar)  
 Mgmt Track: EM → Senior EM → Director → VP → CTO

## Métricas de un Tech Lead Efectivo

| Métrica                | Target                     |
|------------------------|----------------------------|
| PR review turnaround   | < 4 horas                  |
| Sprint velocity (team) | Estable o creciente        |
| Technical debt ratio   | < 15% del sprint           |
| Bus factor del squad   | >= 3                       |
| Onboarding time nuevos | < 2 semanas para primer PR |

## Referencias

- Roles del Equipo

- Gobernanza Técnica
- Prácticas de Ingeniería
- Technical Debt

# Architect

## Software Architect: Rol y Responsabilidades

El Software Architect define la visión técnica de largo plazo, toma decisiones de arquitectura system-wide y guía al equipo en la implementación correcta.

### ¿Qué es un Software Architect?

Un Software Architect es responsable de la **calidad estructural** del sistema de software. Mientras el Tech Lead se enfoca en un squad, el Architect opera a nivel de sistema o portfolio.

"The architect's primary responsibility is to maintain the integrity of the architectural vision." — George Fairbanks, *Just Enough Software Architecture*

## Responsabilidades Core

### Architecture Decisions

- Definir patrones arquitectónicos: microservicios, event-driven, CQRS
- Establecer boundaries entre sistemas y servicios
- Seleccionar tecnologías de infraestructura
- Documentar decisiones en ADR

### Tech Radar

Mantener y evolucionar el tech radar del equipo/organización:

| Quadrant               | Ejemplos                                       |
|------------------------|------------------------------------------------|
| Languages & Frameworks | Kotlin, Spring Boot, React, FastAPI            |
| Platforms              | AWS, Kubernetes, PostgreSQL, Redis             |
| Tools                  | Terraform, GitHub Actions, ArgoCD              |
| Techniques             | Event Sourcing, Circuit Breaker, Strangler Fig |

Cada tecnología se clasifica en:

- Adopt**: listo para uso general
- Trial**: valioso, probar en proyectos reales
- Assess**: investigar y crear POC
- Hold**: no usar en nuevos proyectos

### Design Reviews

- Revisar diseños de arquitectura antes de implementación

- Identificar risks y tradeoffs
- Asegurar compliance con estándares
- Facilitar design reviews formales

## Technology Strategy

- Roadmap técnico de 6-18 meses
- Migraciones y modernización
- Evaluación de build vs buy
- Gestión de technical debt a nivel estratégico

## Decisiones de Arquitectura

### Framework de Decisión

1. Contexto: ¿Qué problema resolvemos?
  2. Options: ¿Cuáles son las alternativas?
  3. Criterios: ¿Qué importa? (cost, time, risk, scalability)
  4. Decision: ¿Qué elegimos y por qué?
  5. Consequences: ¿Qué tradeoffs aceptamos?

Ver ADR para el formato estándar.

### Tipos de Decisiones

| Tipo        | Scope             | Ejemplo                         | Quién decide            |
|-------------|-------------------|---------------------------------|-------------------------|
| Strategic   | Org-wide          | Cloud provider, main tech stack | CTO + Architects        |
| Tactical    | Team/system level | API design, data model          | Architect + Tech Lead   |
| Operational | Feature-level     | Implementation pattern          | Tech Lead + Senior devs |

## Architect vs Tech Lead (Detalle)

| Dimensión       | Software Architect           | Tech Lead            |
|-----------------|------------------------------|----------------------|
| Time horizon    | 1-3 años                     | 1-3 sprints          |
| Scope           | Sistema completo / portfolio | 1 squad              |
| Código          | POCs, prototypes, spikes     | Feature code diario  |
| Stakeholders    | Leadership, múltiples teams  | 1 team + PO          |
| Deliverables    | ADRs, diagrams, tech radar   | Working software     |
| Risk management | Architectural risks          | Implementation risks |

| Dimensión | Software Architect     | Tech Lead          |
|-----------|------------------------|--------------------|
| Knowledge | Breadth (multi-domain) | Depth (team stack) |

## Artefactos del Architect

| Artefacto             | Propósito                         | Frecuencia        |
|-----------------------|-----------------------------------|-------------------|
| Architecture diagrams | Documentar estructura del sistema | On change         |
| ADRs                  | Registrar decisiones y contexto   | Per decision      |
| Tech radar            | Evaluar tecnologías               | Quarterly         |
| Architecture reviews  | Validar diseños                   | Per feature major |
| Capacity planning     | Escalar infra y equipo            | Quarterly         |
| Run books             | Operaciones críticas              | On need           |

Ver Gobernanza Técnica para el proceso completo.

## Modelo de Madurez del Architect

| Nivel   | Capacidad                                      | Ejemplo                              |
|---------|------------------------------------------------|--------------------------------------|
| Nivel 1 | Diseña features dentro de un sistema existente | API design, DB schema                |
| Nivel 2 | Diseña subsistemas completos                   | Payment service, notification engine |
| Nivel 3 | Define arquitectura de sistema multi-service   | E-commerce platform                  |
| Nivel 4 | Arquitectura de portfolio, múltiples productos | Enterprise architecture              |
| Nivel 5 | Define technical strategy org-wide, innova     | Distinguished Engineer               |

## Errores Comunes

| Error                     | Impacto                          | Mitigación                                                   |                            |
|---------------------------|----------------------------------|--------------------------------------------------------------|----------------------------|
| Ivory tower architecture  | Diseños que nadie implementa     | Stay close to code, pair with devs                           |                            |
| Resume-driven development | Elegir tech por hype             | Evaluate pragmatically, ADR                                  |                            |
| Over-engineering          | Complejidad innecesaria          | YAGNI, [[../01-Fundamentos/04-principios-ingenieria-software | principios de ingeniería]] |
| Under-documenting         | Knowledge loss, onboarding lento | Mandatory ADRs for key decisions                             |                            |

| Error           | Impacto                                  | Mitigación                          |
|-----------------|------------------------------------------|-------------------------------------|
| Ignoring Conway | Architecture doesn't match org structure | Consider org design in architecture |

## Herramientas del Architect

| Categoría           | Herramientas                             |
|---------------------|------------------------------------------|
| Diagramming         | Structurizr, PlantUML, Mermaid, C4 model |
| ADR                 | adr-tools, Log4brains, markdown          |
| Tech Radar          | ThoughtWorks Tech Radar, internal tools  |
| Architecture Review | Architecture Fitness Functions, ArchUnit |
| Documentation       | Docusaurus, Notion, Confluence           |

## Referencias

- Tech Lead
- Arquitectura Limpia
- De Monolito a Microservicios
- Team Topologies
- Ley de Conway

# Engineering Manager

# Engineering Manager

El Engineering Manager es responsable de las **personas** y la **entrega** del equipo. Combina people management con execution management.

## ¿Qué es un Engineering Manager?

Un EM maneja personas, no código. Su trabajo es crear el entorno donde los ingenieros puedan hacer su mejor trabajo, alineados con los objetivos del negocio.

"An engineering manager's job is to multiply the output of the team." — Will Larson

## Responsabilidades Core

### People Management

#### 1:1s (Weekly)

| Elemento   | Detalle                               |
|------------|---------------------------------------|
| Frecuencia | Semanal, 30-60 minutos                |
| Propósito  | Career growth, feedback, impedimentos |
| Estructura | 70% persona / 30% proyecto            |
| Output     | Action items, follow-ups, notes       |
| Regla      | El direct report define la agenda     |

#### Template de 1:1:

1. Check-in personal (5 min)
2. Prioridades de la semana (10 min)
3. Bloqueadores y soporte (10 min)
4. Career growth / feedback (10 min)
5. Acciones y next steps (5 min)

### Career Growth

- Definir y mantener career ladders para el equipo
- Crear Individual Development Plans (IDPs)
- Identificar promotion candidates y preparar cases
- Conectar personas con stretch assignments

### Hiring

- Definir job descriptions con PO y Tech Lead
- Diseñar proceso de entrevistas (technical + cultural)
- Participar en entrevistas de culture fit
- Cerrar ofertas y manage expectations
- Diversity & inclusion en el proceso

### Team Health

- Monitorear eNPS (Employee Net Promoter Score)
- Detectar y resolver conflictos temprano
- Prevenir burnout con workload management
- Fomentar psychological safety
- Celebrar logros y recognition

### Execution Management

- Coordinar delivery con PO y Tech Lead
- Remove impedimentos del equipo
- Stakeholder communication
- Sprint/iteration health monitoring
- Risk identification y escalation

### EM vs Tech Lead

| Dimensión            | Engineering Manager     | Tech Lead               |
|----------------------|-------------------------|-------------------------|
| Primary focus        | People & delivery       | Technical quality       |
| Who manages          | People (direct reports) | Code & architecture     |
| 1:1s                 | Yes, weekly             | No (mentoring informal) |
| Hiring               | Full ownership          | Participa en technical  |
| Performance reviews  | Owns process            | Provides input          |
| Code review          | Rarely                  | Daily                   |
| Career conversations | Primary owner           | Supports                |
| Conflict resolution  | Primary owner           | Escalates               |

En equipos pequeños, el EM puede actuar como Tech Lead. Pero a medida que el equipo crece, estos roles deben separarse.

### Modelo de 3 Horizontes

#### H1: Operaciones (60%)

- └─ 1:1s
- └─ Sprint health
- └─ Stakeholder management
- └─ Incident response (people side)

#### H2: Construcción (30%)

- └─ Team culture
- └─ Process improvement
- └─ Hiring pipeline
- └─ Cross-team coordination

#### H3: Estrategia (10%)

- └─ Org design
- └─ Technical strategy (input)
- └─ Budget planning
- └─ Long-term team vision

## Métricas del EM

| Métrica                     | Target         | Cadencia  |
|-----------------------------|----------------|-----------|
| eNPS                        | > 30           | Quarterly |
| Retention rate              | > 90%          | Quarterly |
| Time to fill roles          | < 45 días      | Per hire  |
| Team velocity stability     | < 20% variance | Sprint    |
| Engagement (1:1 completion) | 100%           | Weekly    |
| Career plan coverage        | 100% del team  | Quarterly |

## Team Size Óptimo

| Tamaño del team | Rol del EM                           |
|-----------------|--------------------------------------|
| 3-5 personas    | EM hands-on, possibly IC track       |
| 6-9 personas    | EM full-time, management focus       |
| 10-14 personas  | EM full-time, possibly needs support |
| 15+ personas    | Split team, add EM or TL             |

Amazon y Spotify recomiendan teams de 6-8 personas como óptimo. Más de 10 empieza a perder cohesión.

## Anti-patronos del EM

| Anti-patrón       | Problema                     | Solución                        |
|-------------------|------------------------------|---------------------------------|
| The hero EM       | Hace todo, no delega         | Trust the team, develop leaders |
| The shield EM     | Aísla al equipo de toda info | Filtra, pero no bloque          |
| The micromanager  | Controla cada decisión       | Define standards, da autonomía  |
| The absent EM     | Solo manages calendar        | Dedicate time a people          |
| The promoter      | Solo dice sí al management   | Empower the team to push back   |
| The tech-first EM | Sigue coding, ignora people  | Recognize the role shift        |

## Frameworks Útiles

| Framework              | Uso                                            |
|------------------------|------------------------------------------------|
| Radical Candor         | Feedback: care personally + challenge directly |
| Situational Leadership | Adaptar leadership style al contexto           |
| Top Grading            | Estructurar entrevistas de hiring              |
| OKRs                   | Alineación de objetivos                        |
| Career Ladders         | Framework de crecimiento                       |

## Referencias

- Roles del Equipo
- Organización Típica
- Cultura de Equipo
- Onboarding

---

## Product Owner

# Product Owner / Business Analyst

El Product Owner es el **dueño del backlog** y el puente entre el negocio y el equipo técnico. Define el qué, el equipo define el cómo.

## ¿Qué es un Product Owner?

El PO es responsable de **maximizar el valor** del producto entregado por el equipo. En Software Factories, el PO puede ser interno (dedicado al producto) o externo (client-side).

## Responsabilidades Core

### Backlog Management

| Actividad   | Detalle                               |
|-------------|---------------------------------------|
| Crear items | User stories, technical stories, bugs |
| Refinar     | Estimar, priorizar, split stories     |
| Mantener    | Actualizar con feedback y datos       |
| Comunicar   | Transparente, accesible al equipo     |

## Priorización

### Framework: RICE

| Factor     | Pregunta                   | Score                |
|------------|----------------------------|----------------------|
| Reach      | ¿Cuántos usuarios impacta? | Número               |
| Impact     | ¿Cuánto impacta (1-3)?     | 1=low, 2=med, 3=high |
| Confidence | ¿Qué tan seguros estamos?  | 50-100%              |
| Effort     | ¿Cuánto trabajo cuesta?    | Person-months        |

$$\text{RICE Score} = (\text{Reach} \times \text{Impact} \times \text{Confidence}) / \text{Effort}$$

### Framework: MoSCoW

- **Must have**: blocking, no launch without it
- **Should have**: importante, pero no critical
- **Could have**: nice to have, time permitting
- **Won't have** (this time): explicitly out of scope

## Framework: WSJF (Weighted Shortest Job First)

$WSJF = \text{Cost of Delay} / \text{Job Size}$

Cost of Delay = User Value + Time Criticality + Risk Reduction

## Stakeholder Management

- Recolectar requirements de stakeholders
- Traducir business needs a technical specs
- Manage expectations sobre scope y timeline
- Presentar progress y upcoming priorities
- Negociar tradeoffs con EM y Tech Lead

## User Stories

### Formato estándar:

Como [tipo de usuario]  
Quiero [acción/funcionalidad]  
Para [beneficio/valor]

Criterios de aceptación:

- Dado [contexto], cuando [acción], entonces [resultado]
- Dado [contexto], cuando [acción], entonces [resultado]

Estimación: [story points]

Prioridad: [MoSCoW]

### Ejemplo:

Como comprador online  
Quiero agregar productos al carrito de compras  
Para poder comprar múltiples items en una sola transacción

Criterios de aceptación:

- Dado que estoy en una página de producto, cuando hago clic en "Agregar al carrito", entonces el item se añade con cantidad 1
- Dado que un item ya está en el carrito, cuando lo agrego nuevamente, entonces la cantidad se incrementa en 1
- Dado que el carrito tiene items, cuando voy al checkout, entonces veo el resumen con totales

Estimación: 5 story points

Prioridad: Must have

## PO vs BA en Software Factory

| Aspecto      | Product Owner          | Business Analyst       |
|--------------|------------------------|------------------------|
| Focus        | Valor del producto     | Requisitos del negocio |
| Decide       | Qué se construye       | Cómo se documenta      |
| Backlog      | Owner directo          | Contribute y refine    |
| Stakeholders | Negocio y equipo       | Negocio primariamente  |
| Métrica      | ROI, user satisfaction | Requirements coverage  |
| Ubicación    | Dentro del equipo      | Puede ser shared       |

En muchas Software Factories, los roles de PO y BA se combinan en una sola persona.

## Días Típicos

| Hora  | Actividad                                        |
|-------|--------------------------------------------------|
| 09:00 | Standup con el equipo                            |
| 09:30 | Backlog refinement (story splitting, estimation) |
| 10:30 | Stakeholder meeting (requirements gathering)     |
| 11:30 | User research / feedback review                  |
| 13:00 | Sprint planning prep (stories ready)             |
| 14:00 | Cross-team dependency alignment                  |
| 15:00 | Demo prep / stakeholder demo                     |
| 16:00 | Documentation, acceptance criteria               |
| 16:30 | Sprint review / retrospective support            |

## Métricas del PO

| Métrica                  | Target                   |
|--------------------------|--------------------------|
| Backlog ready ratio      | > 80% de upcoming sprint |
| Story cycle time         | < 3 días                 |
| Feature adoption rate    | > 60%                    |
| Stakeholder satisfaction | > 4/5                    |
| Requirement churn        | < 20% post-planning      |

# Anti-patrones del PO

| Anti-patrón         | Problema                          | Solución                   |
|---------------------|-----------------------------------|----------------------------|
| The order taker     | Solo traduce requests del cliente | Proactive product thinking |
| The feature factory | Más features = más valor          | Outcome-driven, not output |
| The bottleneck      | Todo pasa por el PO               | Empower team, delegate     |
| The absent PO       | Backlog stale, no refinement      | Dedicated time, cadencia   |
| The micromanager    | Define cómo se implementa         | Focus on qué y por qué     |

# Herramientas

| Categoría          | Opciones                              |
|--------------------|---------------------------------------|
| Backlog management | Jira, Linear, Azure DevOps, Shortcut  |
| Prototyping        | Figma (with designer), InVision       |
| User research      | Maze, Hotjar, UserTesting             |
| Roadmapping        | Productboard, Aha!, Roadmunk          |
| Analytics          | Mixpanel, Amplitude, Google Analytics |

# Referencias

- Producto vs Proyecto
- Roles del Equipo
- Procesos Ágiles
- Prácticas de Ingeniería

## Squad Tribe Chapter

# Squads, Tribes, Chapters — Modelo Spotify

El modelo Spotify es un framework organizacional popular en la industria de software. Aunque Spotify lo describe como "inspiración" más que modelo, su estructura se ha replicado ampliamente.

## La Estructura Spotify

### Squad

| Aspecto    | Detalle                                               |
|------------|-------------------------------------------------------|
| Definición | Equipo autónomo, cross-functional, dedicado a un área |
| Tamaño     | 6-12 personas                                         |
| Analogía   | Como un "mini-startup" dentro de la organización      |
| Contiene   | Devs, QA, designers, PO                               |
| Liderazgo  | Squad Lead (puede ser EM o senior engineer)           |
| Objetivo   | Ship value de forma autónoma                          |

### Tribe

| Aspecto    | Detalle                                     |
|------------|---------------------------------------------|
| Definición | Colección de squads relacionados            |
| Tamaño     | 40-150 personas                             |
| Analogía   | Como un "campus" de startups                |
| Contiene   | Múltiples squads con dominio similar        |
| Liderazgo  | Tribe Lead                                  |
| Objetivo   | Coordinación entre squads, shared alignment |

### Chapter

| Aspecto    | Detalle                                                |
|------------|--------------------------------------------------------|
| Definición | Grupo de personas con el mismo skill set across squads |
| Ejemplos   | Chapter de Backend, Chapter de QA, Chapter de Data     |

| Aspecto   | Detalle                                            |
|-----------|----------------------------------------------------|
| Contiene  | Engineers del mismo specialty de diferentes squads |
| Liderazgo | Chapter Lead (línea funcional)                     |
| Objetivo  | Best practices, career growth, hiring standards    |

## Guild

| Aspecto    | Detalle                                          |
|------------|--------------------------------------------------|
| Definición | Comunidad de interés, transversal                |
| Ejemplos   | Guild de security, guild de AI, guild de testing |
| Contiene   | Miembros voluntarios de cualquier tribe/squad    |
| Liderazgo  | Community-driven, sin management formal          |
| Objetivo   | Knowledge sharing, innovation, standards         |

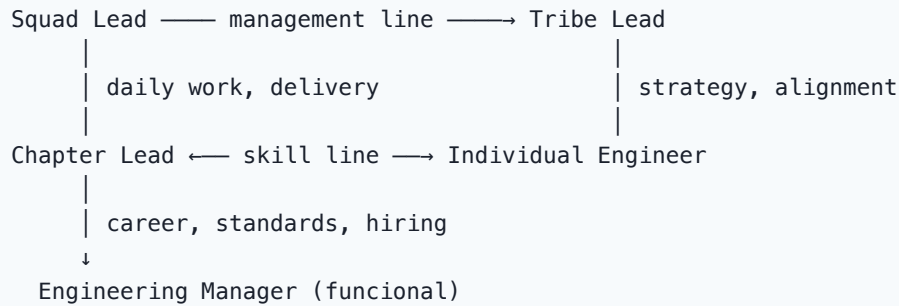
## Diagrama de Estructura

```
Tribe A (Payments)
├── Squad: Checkout
│   ├── 3 Backend devs (Chapter Backend)
│   ├── 2 Frontend devs (Chapter Frontend)
│   ├── 1 QA (Chapter Quality)
│   └── 1 Designer
├── Squad: Fraud Detection
│   ├── 2 Backend devs
│   ├── 1 ML engineer
│   └── 1 Data analyst
└── Squad: Billing
    ├── 2 Backend devs
    └── 1 QA

Chapter Backend (across all squads)
├── Chapter Lead: Senior Backend
├── Standards: Java, Spring Boot, testing patterns
└── Career: IC track, mentorship

Guild: AI/ML (across all tribes)
├── Monthly meetup
├── Shared tooling
└── Knowledge base
```

## Líneas de Reporte Dueling



#### Doble reporte:

- **Management line:** Squad Lead → Tribe Lead (delivery focus)
- **Skill line:** Chapter Lead → Chapter members (career/standards focus)

## Ventajas del Modelo Spotify

| Ventaja           | Descripción                           |
|-------------------|---------------------------------------|
| Autonomía         | Squads deciden cómo trabajar          |
| Ownership         | Cada squad owns su dominio            |
| Alignment         | Tribes agrupan esfuerzos relacionados |
| Career growth     | Chapters proporcionan path funcional  |
| Knowledge sharing | Guilds diseminan innovación           |
| Scalability       | Se puede replicar el patrón           |

## Desventajas y Críticas

| Desventaja             | Descripción                                   |
|------------------------|-----------------------------------------------|
| Complejidad de reporte | Doble línea puede generar confusión           |
| Cultural dependency    | Requiere alta madurez ágil                    |
| No es un template      | Spotify dice "no nos copien"                  |
| Coordination cost      | Cross-squad dependencies son difíciles        |
| Role ambiguity         | Squad Lead vs EM vs Chapter Lead se superpone |

En 2020, Spotify mismo reconoció que su modelo ha evolucionado significativamente y no es tan "flat" como se presenta.

## Cuándo Usar Este Modelo

| Contexto                             | Recomendación                     |
|--------------------------------------|-----------------------------------|
| SF con 50+ ingenieros                | Buen ajuste                       |
| SF con 10-30 ingenieros              | Considerar version simplificada   |
| Organización con múltiples productos | Tribe = producto, Squad = feature |
| Organización matricial existente     | Puede ser difícil migrar          |
| Startup pequeña                      | Over-engineering organizacional   |

## Modelo Simplificado para SFs Pequeñas

Para una Software Factory de 20-50 personas:

```
Engineering Director
├─ Squads (3-4 personas cada uno)
│   ├── Squad 1: Product A
│   ├── Squad 2: Product B
│   └── Squad 3: Platform
├─ Tech Leads (comparten chapter duties)
└─ Guilds informales (weekly tech talks)
```

## Comparación con Otros Modelos

| Modelo             | Estructura             | Autonomía | Complejidad |
|--------------------|------------------------|-----------|-------------|
| Spotify            | Squad/Tribe/Chapter    | Alta      | Alta        |
| Team Topologies    | 4 tipos de equipo      | Media     | Media       |
| Amazon Two-Pizza   | Small autonomous teams | Alta      | Baja        |
| Traditional Matrix | Funcional + proyectos  | Baja      | Media       |

Ver Team Topologies y Estructura Matricial.

## Referencias

- Equipos Autónomos
- Organización Típica
- Team Topologies
- Ley de Conway
- Cultura de Equipo

# Gobernanza Tecnica

## Gobernanza Técnica

Sistemas y procesos para tomar decisiones técnicas de forma consistente, transparente y escalable en una organización de software.

### ¿Qué es la Gobernanza Técnica?

La gobernanza técnica es el framework que define **cómo** se toman decisiones técnicas en la organización. No es burocracia — es sobre crear claridad, consistencia y trazabilidad.

### Componentes Principales

#### 1. Tech Council

| Aspecto    | Detalle                                                         |
|------------|-----------------------------------------------------------------|
| Qué es     | Grupo de senior engineers que revisan decisiones técnicas clave |
| Quién      | Staff+ engineers, Architects, Senior Tech Leads                 |
| Frecuencia | Quincenal o mensual                                             |
| Scope      | Decisiones de alcance multi-team                                |

Agenda típica de Tech Council:

1. Review de ADRs pendientes (15 min)

2. Tech radar updates (10 min)

3. Architecture review de proyecto importante (20 min)

4. Technical debt prioritization (10 min)

5. Cross-cutting concerns (security, performance) (10 min)

6. New technology evaluations (5 min)

#### 2. RFC (Request for Comments)

Proceso formal para propuestas técnicas significativas:

| Etapas      | Descripción                            | Owner                    |
|-------------|----------------------------------------|--------------------------|
| Draft       | Author escribe la propuesta            | Cualquier engineer       |
| Review      | Feedback de stakeholders (1-2 semanas) | Team + Tech Council      |
| Decision    | Aprobación/rechazo con rationale       | Tech Council / Architect |
| Implemented | La propuesta se ejecuta                | Author + team            |

| Etapa  | Descripción             | Owner             |
|--------|-------------------------|-------------------|
| Closed | Rechazada o supersedida | Con documentación |

Template de RFC:

```
# RFC: [Título]

## Status
Draft | Review | Approved | Rejected | Superseded

## Summary
Una或 dos oraciones sobre qué propone.

## Motivation
¿Por qué es necesario? ¿Qué problema resuelve?

## Detailed Design
Descripción técnica detallada.

## Alternatives Considered
Otras opciones evaluadas.

## Migration Plan
Cómo migrar del estado actual al nuevo.

## Rollback Plan
Cómo revertir si algo sale mal.

## Open Questions
Preguntas que necesitan resolver antes de implementar.
```

3. Design Reviews

| Tipo                | Cuándo                                | Participantes                 |
|---------------------|---------------------------------------|-------------------------------|
| Feature design      | Antes de implementar feature compleja | Squad + Architect             |
| Architecture review | Cambios system-wide                   | Tech Council + affected teams |
| Security review     | Features con datos sensibles          | Security + Squad              |
| Performance review  | Cambios con impacto de performance    | SRE + Squad                   |
| Post-mortem         | Después de incidente mayor            | Involved teams + management   |

4. Tech Radar

Actualización trimestral del tech radar:

| Quadrant       | Acción requerida                    |
|----------------|-------------------------------------|
| Move to Adopt  | Requiere aprobación de Tech Council |
| Move to Trial  | Tech Lead puede decidir             |
| Move to Assess | Cualquier engineer puede proponer   |
| Move to Hold   | Requiere plan de migración          |

Ver Software Architect para más sobre tech radar.

## Proceso de Decisión Técnica

```
Propuesta (cualquiera)
  ↓
RFC/ADR escrito
  ↓
Review por team affected
  ↓
Escalation (si es cross-team)
  ↓
Tech Council decision
  ↓
Documentado (ADR o RFC)
  ↓
Implementado
  ↓
Follow-up review
```

## Decisiones que Requieren Gobernanza

| Tipo de Decisión     | Nivel de Gobernanza       | Ejemplo                     |
|----------------------|---------------------------|-----------------------------|
| Technology adoption  | Tech Council              | Nuevo framework frontend    |
| Architecture pattern | Architect + Tech Council  | Event-driven architecture   |
| Database choice      | Tech Council              | PostgreSQL vs MongoDB       |
| CI/CD approach       | Platform team + Tech Lead | GitHub Actions vs GitLab CI |
| Coding standards     | Chapter leads             | Style guide, patterns       |
| Library choice       | Tech Lead                 | Testing framework           |

## Gobernanza Ligera vs Pesada

| Contexto           | Ligera                             | Pesada                    |
|--------------------|------------------------------------|---------------------------|
| Startup/SF pequeña | RFC informal, Slack thread         | —                         |
| SF mediana         | RFC template, Tech Council mensual | —                         |
| SF grande          | Formal RFC process                 | Architecture review board |
| Enterprise         | Full governance framework          | Compliance, audit trails  |

El objetivo es encontrar el **mínimo proceso necesario** para mantener consistencia sin ralentizar la entrega.

## Métricas de Gobernanza

| Métrica                  | Target                 | Cadencia    |
|--------------------------|------------------------|-------------|
| RFC turnaround           | < 2 semanas            | Per RFC     |
| ADR coverage             | 100% de decisiones key | Quarterly   |
| Tech debt ratio          | < 15% del capacity     | Sprint      |
| Tech radar updates       | Quarterly              | On schedule |
| Design review completion | > 90%                  | Per review  |

## Anti-patrones de Gobernanza

| Anti-patrón               | Problema                             | Solución                     |
|---------------------------|--------------------------------------|------------------------------|
| Rubber stamp council      | Aprobar todo sin review              | Require written feedback     |
| Decision by loudest voice | Dominación de personalidad           | Structured decision process  |
| Governance theater        | Proceso sin valor real               | Measure outcome, not process |
| Ivory tower               | Decisiones disconnected from reality | Involve implementers         |
| No documentation          | Decisiones se pierden                | Mandatory ADRs               |

## Referencias

- ADR
- Software Architect
- Tech Lead
- Equipos Autónomos
- Calidad y Seguridad

## Adr Decisiones

# Architecture Decision Records (ADR)

Los ADRs son documentos que capturan decisiones arquitectónicas importantes: qué se decidió, por qué, y qué tradeoffs se aceptaron.

## ¿Qué es un ADR?

Un Architecture Decision Record es un documento de una sola página que registra una decisión arquitectónica significativa, su contexto, y sus consecuencias.

"ADR = el periódico de la arquitectura" — documenta lo que pasó y por qué, antes de que el conocimiento se pierda.

## Por Qué Usar ADRs

| Beneficio            | Descripción                                       |
|----------------------|---------------------------------------------------|
| Trazabilidad         | Saber por qué se tomó una decisión específica     |
| Onboarding           | Nuevos miembros entienden el contexto histórico   |
| Evitar re-discussion | No revisitar decisiones ya tomadas sin nueva info |
| Accountability       | Queda claro quién decidió y cuándo                |
| Comunicación         | Alinear a múltiples equipos sobre decisiones      |
| Preservar contexto   | Cuando el equipo que decidió ya no está           |

## Formato Estándar

### Template: Michael Nygard (El más usado)

```
# ADR [ número]: [Título de la Decisión]

## Estado
[Propuesto | Aceptado | Obsoleto | Reemplazado por ADR-XXX]

## Contexto
[Describe el contexto y forces que llevaron a esta decisión.
No es solo technical – incluye business, team, timeline.]

## Decisión
[Describe la decisión tomada de forma clara y concisa.
Usa voz activa: "Decidimos..."]
```

```
## Consecuencias
[Que se gana, que se pierde, y qué tradeoffs se aceptan.]

### Positivas
- [Beneficio 1]
- [Beneficio 2]

### Negativas
- [Costo 1]
- [Costo 2]

### Neutrales
- [Impacto que es neither positive nor negative]
```

## Template Avanzado (para decisiones complejas)

```
# ADR [number]: [Title]

## Estado
Propuesto | Aceptado | Obsoleto | Reemplazado por [[ADR-XXX]]

## Fecha
YYYY-MM-DD

## Contexto
### Problema
[¿Qué problema resuelve esta decisión?]

### Drivers
- [Driver 1: business requirement, technical constraint, etc.]
- [Driver 2]
- [Driver 3]

### Constraints
- [Constraint 1: budget, time, team, regulation]
- [Constraint 2]

## Options Considered

### Option A: [Nombre]
**Pros:** ...
**Cons:** ...
**Riesgo:** ...

### Option B: [Nombre]
**Pros:** ...
**Cons:** ...
**Riesgo:** ...

### Option C: [Nombre]
```

```
**Pros:** ...
**Cons:** ...
**Riesgo:** ...

## Decision
Decidimos implementar **Option [X]** because [rationale].

## Consecuencias
### Esperadas
- [Consequence 1]
- [Consequence 2]

### Riesgos
- [Risk 1] – Mitigación: [plan]

## Follow-up Actions
- [ ] [Action 1] – Owner: [person] – Due: [date]
- [ ] [Action 2] – Owner: [person] – Due: [date]

## References
- [[ADR-XXX]] (superseded by this)
- [Link to RFC if applicable]
```

## Convención de Numeración

```
docs/adr/
├─ 0001-use-postgresql-as-primary-db.md
├─ 0002-adopt-event-driven-architecture.md
├─ 0003-use-kubernetes-for-orchestration.md
├─ 0004-implement-cqrs-for-order-service.md
└─ 0005-migrate-from-rest-to-graphql.md
```

### Reglas:

- Números secuenciales, zero-padded a 4 dígitos
- Nombres descriptivos en kebab-case
- Una carpeta docs/adr/ en la raíz del repo
- Solo se crean, nunca se borran

## Ejemplos Reales

### ADR 0001: PostgreSQL como Base de Datos Principal

```
# ADR 0001: PostgreSQL como Base de Datos Principal

## Estado
Aceptado
```

### ## Contexto

Necesitamos elegir una base de datos relacional para el sistema principal de órdenes. El equipo tiene experiencia con PostgreSQL y MySQL. Necesitamos ACID compliance, JSON support, y extensibilidad.

### ## Decisión

Usaremos PostgreSQL como base de datos principal.

### ## Consecuencias

#### ### Positivas

- JSON/JSONB para datos semi-estructurados
- Extensibilidad (PostGIS, pg\_trgm)
- Comunidad activa y ecosistema maduro
- Licencia permisiva

#### ### Negativas

- Curva de aprendizaje para operaciones de DBA
- Menor disponibilidad de hosting managed vs MySQL

## ADR 0002: Event-Driven para Comunicación entre Servicios

# ADR 0002: Event-Driven Architecture para Communication

### ## Estado

Aceptado

### ## Contexto

Tenemos 5 microservicios que necesitan comunicarse.

El request-response directo genera acoplamiento fuerte y cascade failures.

### ## Decisión

Usaremos event-driven architecture con Apache Kafka para comunicación asincrónica entre servicios.

### ## Consecuencias

#### ### Positivas

- Desacoplamiento de servicios
- Resiliencia: los consumers fallan independientemente
- Scalability horizontal de event processing

#### ### Negativas

- Complejidad de debugging distribuido
- eventual consistency (no strong consistency)
- Requiere infraestructura adicional (Kafka cluster)

## Herramientas para ADRs

| Herramienta       | Tipo          | Ventaja                     |
|-------------------|---------------|-----------------------------|
| adr-tools         | CLI           | Init, list, new, supersedes |
| Log4brains        | Web UI        | Visualización y navegación  |
| Markdown files    | Manual        | Simple, version controlled  |
| Notion/Confluence | Collaborative | Editable, commentable       |
| GitHub/GitLab     | PR-based      | Review process, approval    |

## Uso de adr-tools

```
# Instalar
npm install -g adr-tools

# Inicializar en un repo
adr init

# Crear nuevo ADR
adr new "Use PostgreSQL as primary database"

# Listar ADRs
adr list

# Referenciar un ADR
adr link 1 superseded-by 3
```

## ADR vs RFC

| Aspecto     | ADR                       | RFC                               |
|-------------|---------------------------|-----------------------------------|
| Scope       | 1 decisión arquitectónica | Propuesta amplia                  |
| Audience    | Técnico, largo plazo      | Técnico + stakeholders            |
| Vida útil   | Permanente                | Temporal (se implementa o cierra) |
| Formato     | Más corto, estructurado   | Más largo, incluye alternativas   |
| Cuándo usar | Decisión tomada           | Propuesta en review               |

Ver Gobernanza Técnica para el proceso completo.

## ADRs y AI

Con IA, los ADRs pueden ser asistidos por:

- **Generación de draft:** AI genera borrador basado en requirements
- **Review assistance:** AI identifica gaps en el ADR

- **Decision impact analysis:** AI evalúa consecuencias potenciales
- **Historical search:** AI encuentra ADRs relevantes para decisions actuales

## Referencias

- Gobernanza Técnica
- Software Architect
- Technical Debt
- Arquitectura Limpia

## Equipos Autonomos

# Equipos Autónomos y Auto-organizados

La autonomía del equipo es un pilar del ágil moderno pero requiere balance entre libertad y alineación organizacional.

## ¿Qué es un Equipo Autónomo?

Un equipo autónomo tiene **decisión sobre cómo, qué, y parcialmente cuándo** trabajar, dentro de constraints definidos por la organización.

"Autonomy does not mean working in isolation or doing whatever you want. It means the freedom to decide how to best achieve the goals set by the organization." — Marty Cagan

## Niveles de Autonomía

| Nivel   | Scope de Decisión                           | Ejemplo                          |
|---------|---------------------------------------------|----------------------------------|
| Nivel 1 | Cómo implementar (design & code)            | Equipo decide patrones de código |
| Nivel 2 | Qué features construir (dentro del roadmap) | Equipo prioriza backlog propio   |
| Nivel 3 | Cómo organizar el trabajo (process)         | Equipo elige framework ágil      |
| Nivel 4 | Qué tecnología usar (within guardrails)     | Equipo selecciona librerías      |
| Nivel 5 | Qué producto construir (full product)       | Equipo define roadmap propio     |

## Matriz de Autonomía vs Alineación



**Sweet spot:** Autonomía controlada — dirección clara + libertad de ejecución.

# Modelos de Autonomía

## Amazon "Two-Pizza Teams"

| Aspecto         | Detalle                                  |
|-----------------|------------------------------------------|
| Tamaño          | 6-8 personas (dos pizzas)                |
| Autonomía       | Full stack ownership                     |
| Responsabilidad | Build, run, own their service            |
| Alignment       | Through API contracts y business metrics |
| Decisiones      | Team-level, without approval             |

## Netflix "Freedom & Responsibility"

| Aspecto   | Detalle                             |
|-----------|-------------------------------------|
| Principio | "Context, not control"              |
| Autonomía | Alta — ingenieros deciden           |
| Alignment | Through culture y business context  |
| Hiring    | "Keeper test" — solo los mejores    |
| Risk      | High autonomy = high accountability |

## Toyota "Lean Teams"

| Aspecto     | Detalle                                          |
|-------------|--------------------------------------------------|
| Principio   | Continuous improvement, respect for people       |
| Autonomía   | En la mejora de process, no en product direction |
| Alignment   | Through hoshin kanri (policy deployment)         |
| Improvement | Kaizen events, suggestion system                 |

# Guardrails: La Contraparte de la Autonomía

La autonomía sin guardrails es caos. Los guardrails definen los límites dentro de los cuales el equipo es libre.

| Guardrail    | Ejemplo                                      | Quién define    |           |
|--------------|----------------------------------------------|-----------------|-----------|
| Security     | No secrets in code, security review required | Security team   |           |
| Architecture | Follow [../02-Estructura/09-adr-decisiones   | ADR]] standards | Architect |

| Guardrail  | Ejemplo                              | Quién define                             |                |
|------------|--------------------------------------|------------------------------------------|----------------|
| Quality    | Minimum test coverage 80%            | [[../02-Estructura/08-gobernanza-tecnica | Tech Council]] |
| Compliance | GDPR, SOC2 requirements              | Legal + Compliance                       |                |
| Budget     | Cloud spend within allocated budget  | Finance + EM                             |                |
| Timeline   | Delivery commitments to stakeholders | PO + EM                                  |                |

## Guardrails vs Gates

| Concepto  | Definición                           | Ejemplo                         |
|-----------|--------------------------------------|---------------------------------|
| Guardrail | Límites que el equipo no debe cruzar | "No deploy without tests"       |
| Gate      | Checkpoint formal antes de avanzar   | "Security review before launch" |

Los guardrails son **preventivos**; los gates son **detectivos**.

## Alineación sin Micromanagement

### OKRs como Mecanismo de Alineación

```
Company OKR: Increase customer retention by 20%

├─ Tribe OKR (Payments): Reduce checkout friction
│   ├── Squad 1 OKR: One-click checkout
│   ├── Squad 2 OKR: Multiple payment methods
│   └── Squad 3 OKR: Auto-retry failed payments
└─ Tribe OKR (Platform): Improve performance
    ├── Squad 4 OKR: API response < 100ms
    └── Squad 5 OKR: 99.99% uptime
```

Ver Gestión de Proyectos para OKRs detallados.

## Communication Cadences

| Cadencia      | Propósito             | Participantes |
|---------------|-----------------------|---------------|
| Squad standup | Daily sync            | Squad         |
| Squad retro   | Process improvement   | Squad         |
| Tribe sync    | Cross-squad alignment | Tribe leads   |
| Chapter meet  | Skill alignment       | Chapter       |

| Cadencia  | Propósito            | Participantes |
|-----------|----------------------|---------------|
| All-hands | Company-wide updates | Everyone      |

## Métricas de Autonomía Saludable

| Métrica                  | Target                            | Señal de Alerta |
|--------------------------|-----------------------------------|-----------------|
| Deployment frequency     | > 1/día por squad                 | < 1/mes         |
| Decision latency         | < 24h para decisiones squad-level | > 1 semana      |
| Team satisfaction        | > 4/5                             | < 3/5           |
| Cross-squad dependencies | < 20% del sprint                  | > 50%           |
| Escalation rate          | < 5% de decisiones                | > 30%           |

## Anti-patrones

| Anti-patrón             | Problema                                         | Solución                     |
|-------------------------|--------------------------------------------------|------------------------------|
| Autonomy theater        | Se dice autónomo pero todo requiere approval     | Audit decision processes     |
| Chaos engineering (org) | Demasiada autonomía, sin alignment               | Define guardrails, OKRs      |
| Dependency hell         | Equipos autónomos crean interfaces incompatibles | Platform team, API standards |
| Tribal knowledge        | Cada squad reinventa la rueda                    | Guilds, shared standards     |
| Blame culture           | Autonomía sin safety                             | Psychological safety first   |

## Referencias

- Squads, Tribes, Chapters
- Team Topologies
- Engineering Manager
- Manifiesto Ágil
- Cultura de Equipo

## Estructura Matricial

# Estructura Matricial vs Funcional vs por Producto

Tres patrones fundamentales de organización para Software Factories, cada uno con tradeoffs diferentes.

## Visión General

| Patrón       | Principio                      | Ejemplo                              |
|--------------|--------------------------------|--------------------------------------|
| Funcional    | Agrupar por especialidad       | Todos los backend en un departamento |
| Por Producto | Agrupar por dominio de negocio | Equipo dedicado a pagos              |
| Matricial    | Combinar ambos                 | Ingenieros reportan a EM + Tech Lead |
| Spotify      | Squad/Tribe/Chapter            | Híbrido con doble reporte            |

## 1. Estructura Funcional

### Diagrama

```
graph TD
    VP[VP Engineering] --> DB[Director Backend]
    VP --> DF[Director Frontend]
    VP --> DQA[Director QA]
    VP --> DDO[Director DevOps]
    DB --> TBA[Team Backend A]
    DB --> TBB[Team Backend B]
    DB --> TBC[Team Backend C]
    DF --> TW[Team Web]
    DF --> TM[Team Mobile]
    DF --> TD[Team Desktop]
    DQA --> TMQA[Team Manual QA]
    DQA --> TQA[Team Automation]
    DDO --> TCI[Team CI/CD]
    DDO --> TI[Team Infrastructure]
```

## Características

| Aspecto      | Detalle                                  |
|--------------|------------------------------------------|
| Reporting    | Por especialidad técnica                 |
| Coordination | Cross-team projects, matrix coordination |
| Career path  | Clara dentro de cada función             |

| Aspecto           | Detalle                   |
|-------------------|---------------------------|
| Knowledge sharing | Alto dentro de la función |

## Ventajas

- Profundidad técnica (everyone with same skills together)
- Career progression clara (junior → senior → lead)
- Best practices compartidas fácilmente
- Hiring y onboarding por skill set

## Desventajas

- **Silos**: comunicación lenta entre funciones
- **Coordination overhead**: proyectos requieren múltiples departamentos
- **Ownership diluido**: nadie es dueño del producto completo
- **Delivery lento**: pasar features entre departments = handoffs

## Cuándo usar

- Organizaciones con pocos productos pero alta complejidad técnica
- Cuando la especialización es más importante que la velocidad
- Enterprise con compliance requirements por función

## 2. Estructura por Producto

### Diagrama

```

VP Product
├── Director Payments Product
│   ├── Backend devs (pagos)
│   ├── Frontend devs (pagos)
│   ├── QA (pagos)
│   ├── Designer (pagos)
│   └── PO (pagos)
├── Director Catalog Product
│   ├── Backend devs (catálogo)
│   ├── Frontend devs (catálogo)
│   ├── QA (catálogo)
│   └── PO (catálogo)
└── Director Platform Product
    ├── Platform engineers
    ├── SRE
    └── PO (platform)
  
```

## Características

| Aspecto           | Detalle                                         |
|-------------------|-------------------------------------------------|
| Reporting         | Por producto/dominio                            |
| Coordination      | Cross-product concerns handled by platform team |
| Career path       | Menos clara (generalists over specialists)      |
| Knowledge sharing | Within product team es alto, cross-team es bajo |

## Ventajas

- **Ownership total:** el equipo es dueño del producto
- **Speed:** sin handoffs, decisiones rápidas
- **Alignment:** team entiende el business context
- **Autonomy:** el equipo puede iterar independientemente

## Desventajas

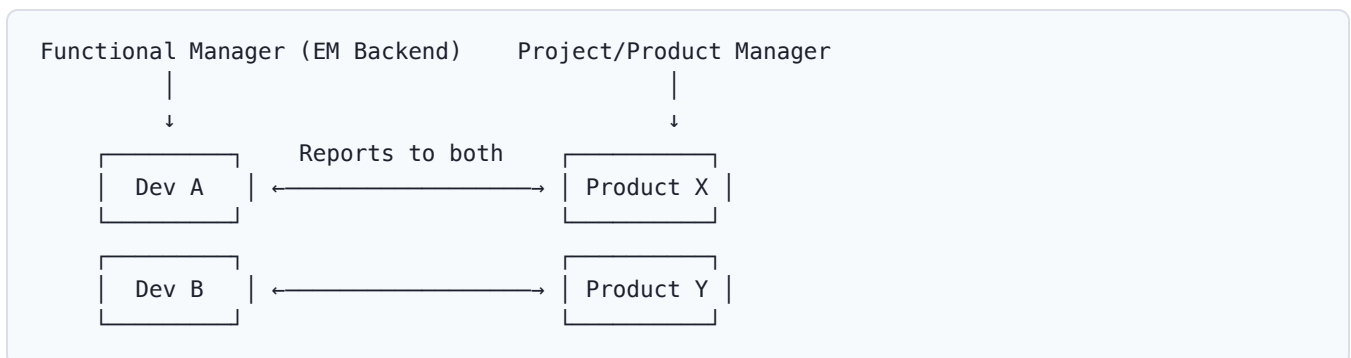
- **Duplicación:** cada equipo puede reinventar soluciones
- **Inconsistent standards:** cada equipo tiene sus patterns
- **Career:** más difícil para especialistas crecer técnicamente
- **Resource waste:** duplication de expertise

## Cuándo usar

- Organizaciones con múltiples productos distintos
- Cuando time-to-market es crítico
- Cuando el business value es más importante que la eficiencia técnica

## 3. Estructura Matricial

### Diagrama



### Características

| Aspecto      | Detalle                             |
|--------------|-------------------------------------|
| Reporting    | Dual: funcional + proyecto/producto |
| Coordination | Through the matrix structure        |
| Career path  | Clara en ambas dimensiones          |
| Flexibilidad | Alta — recursos compartidos         |

## Ventajas

- **Flexibilidad:** recursos se mueven entre proyectos
- **Utilización:** people can work on multiple things
- **Dual expertise:** technical depth + product knowledge
- **Resource efficiency:** no duplication de roles

## Desventajas

- **Conflicting priorities:** dos jefes = dos prioridades
- **Complexity:** difícil de manejar
- **Political:** 谁 tiene más influence
- **Overhead:** más management needed

## Cuándo usar

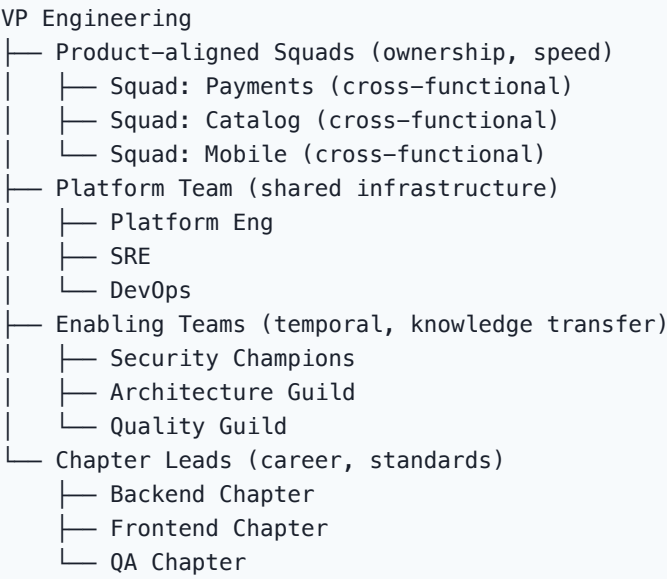
- Organizaciones con recursos limitados que sirven múltiples productos
- Cuando la flexibilidad de asignación es prioritaria
- Transiciones entre modelos organizacionales

## 4. Comparación Directa

| Criterio              | Funcional | Por Producto   | Matricial | Spotify        |
|-----------------------|-----------|----------------|-----------|----------------|
| Velocidad de delivery | Baja      | Alta           | Media     | Alta           |
| Ownership             | Bajo      | Alto           | Medio     | Alto           |
| Especialización       | Alta      | Baja           | Media     | Media          |
| Flexibilidad          | Baja      | Media          | Alta      | Media          |
| Complejidad org       | Baja      | Baja           | Alta      | Alta           |
| Escalabilidad         | Media     | Alta           | Media     | Alta           |
| Career clarity        | Alta      | Baja           | Media     | Media          |
| Conway alignment      | Monolito  | Microservicios | Híbrido   | Microservicios |

# Modelo Híbrido Recomendado para SFs

La mayoría de Software Factories exitosas usan un **híbrido**:



## Decision Framework

| Si necesitas...          | Elige...               |
|--------------------------|------------------------|
| Deep technical expertise | Funcional              |
| Fast product delivery    | Por producto           |
| Resource flexibility     | Matricial              |
| Balance entre todos      | Híbrido (Spotify-like) |

## Referencias

- Organización Típica
- Ley de Conway
- Team Topologies
- Squads, Tribes, Chapters
- Equipos Autónomos

## Onboarding Equipo

# Onboarding de Nuevo Talento

Un buen onboarding reduce el time-to-productivity de 6 meses a 6 semanas y mejora drásticamente la retención.

## ¿Por Qué Importa el Onboarding?

| Métrica              | Sin onboarding formal | Con onboarding formal |
|----------------------|-----------------------|-----------------------|
| Time to first PR     | 3-4 semanas           | 3-5 días              |
| Time to productive   | 6 meses               | 6-8 semanas           |
| Retención a 1 año    | 60-70%                | 85-95%                |
| eNPS de nuevos       | < 10                  | > 30                  |
| Manager satisfaction | Baja                  | Alta                  |

El 28% de nuevos empleados renuean en el primer año si el onboarding es pobre (SHRM).

## Plan 30-60-90 Días

### Días 1-30: Aprender

| Semana   | Objetivo               | Actividades                                                 |                          |
|----------|------------------------|-------------------------------------------------------------|--------------------------|
| Semana 1 | Setup y context        | Computer setup, access, team intro, company overview        |                          |
| Semana 2 | Herramientas y process | CI/CD, repos, coding standards, agile process               |                          |
| Semana 3 | Primer contribution    | Pair programming, small bug fix, first PR                   |                          |
| Semana 4 | Deep dive              | Architecture overview, [.../02-Estructura/09-adr-decisiones | ADRs]], domain knowledge |

### Deliverables del primer mes:

- ☐ Environment de desarrollo funcionando
- ☐ Primer PR merged (bug fix o small feature)
- ☐ Conoce a todos los miembros del equipo
- ☐ Entiende el product domain
- ☐ Completó security training

## Días 31-60: Contribuir

| Semana     | Objetivo          | Actividades                                        |
|------------|-------------------|----------------------------------------------------|
| Semana 5-6 | Feature ownership | Own una feature small-to-medium de principio a fin |
| Semana 7-8 | Team integration  | Participar en design discussions, sprint planning  |

### Deliverables del segundo mes:

- ☐ Feature completa merged a production
- ☐ Participa activamente en code reviews
- ☐ Entiende los processes de gobernanza
- ☐ Ha hecho al menos 1 pair programming session
- ☐ Conoce las key ADRs del proyecto

## Días 61-90: Ownership

| Semana       | Objetivo         | Actividades                                        |
|--------------|------------------|----------------------------------------------------|
| Semana 9-10  | Independence     | Leading small features, making technical decisions |
| Semana 11-12 | Full integration | Part of all team ceremonies, contributing ideas    |

### Deliverables del tercer mes:

- ☐ Feature compleja entregada independientemente
- ☐ Ha presentado en un tech talk o demo
- ☐ Participa en hiring (interviews)
- ☐ Tiene un IDP con su EM
- ☐ Se siente como miembro integral del equipo

## Buddy System

### ¿Qué es un Buddy?

Un buddy es un miembro senior del equipo asignado para guiar al nuevo durante las primeras semanas.

| Aspecto    | Detalle                          |
|------------|----------------------------------|
| Quién      | Senior engineer del mismo equipo |
| Duración   | 30-90 días                       |
| Commitment | 2-4 horas/semana                 |
| Relación   | No es manager, es guía y mentor  |

### Responsabilidades del Buddy

| Semana      | Responsabilidades                               |
|-------------|-------------------------------------------------|
| Semana 1    | Setup, tour, introductions, answered questions  |
| Semana 2-4  | Pair programming, code reviews, context sharing |
| Semana 5-8  | Feature guidance, architecture explanations     |
| Semana 9-12 | Delegation, trust building, check-ins           |

## Reglas para Buddies

1. **No hacer el trabajo** del nuevo — guiar, no resolver
2. **Preguntar, no asumir** — "¿Necesitas ayuda con...?"
3. **Feedback honesto** — positivo y constructivo
4. **Disponibilidad** — ser accessible pero no 24/7
5. **Retrospective** — feedback al EM sobre el proceso

## Documentación de Onboarding

### Checklist para el Equipo

#### Pre-arrival (antes del día 1):

- ☐ Laptop ordenado y configurado
- ☐ Accounts creados (email, Slack, GitHub, Jira, VPN)
- ☐ Welcome email con first-day instructions
- ☐ Buddy asignado
- ☐ Team notified

#### Day 1:

- ☐ Welcome meeting con EM
- ☐ Tour de la office / virtual office
- ☐ Team introductions (1:1 brief)
- ☐ Setup checklist
- ☐ First lunch con buddy

#### Week 1:

- ☐ Access a todos los repos necesarios
- ☐ Development environment running
- ☐ CI/CD pipeline understood
- ☐ First PR submitted
- ☐ Security training completado
- ☐ Coding standards review

## Knowledge Base de Onboarding

Crear una sección en el KB del equipo:

```
team-wiki/
├── onboarding/
│   ├── checklist.md
│   ├── day-1-guide.md
│   ├── week-1-guide.md
│   ├── architecture-overview.md
│   ├── coding-standards.md
│   ├── useful-links.md
│   └── glossary.md
├── architecture/
├── processes/
└── decisions/ (ADRs)
```

Ver Internal Developer Platform para self-service onboarding.

## Métricas de Onboarding

| Métrica                     | Target      | Cadencia   |
|-----------------------------|-------------|------------|
| Time to first PR            | < 5 días    | Per hire   |
| Time to productive          | < 6 semanas | Per hire   |
| Buddy satisfaction (survey) | > 4/5       | Day 30     |
| New hire satisfaction       | > 4/5       | Day 30, 90 |
| 90-day retention            | 100%        | Quarterly  |
| Onboarding NPS              | > 50        | Per cohort |

## Errores Comunes

| Error                     | Impacto                 | Solución                        |
|---------------------------|-------------------------|---------------------------------|
| Overwhelming first day    | Anxiety, disengagement  | Gradual information load        |
| No buddy                  | Isolation, slow ramp    | Always assign buddy             |
| Sink or swim              | Fear, mistakes          | Structured guidance             |
| No feedback loop          | Process never improves  | Regular retrospectives          |
| Documentation void        | Every new hire restarts | Maintain living docs            |
| Only technical onboarding | Cultural disconnect     | Include culture, values, people |

## Onboarding Remoto

| Aspecto       | Adaptación                                          |
|---------------|-----------------------------------------------------|
| Communication | More async, more documentation                      |
| Buddy         | Daily check-ins first week, then weekly             |
| Social        | Virtual coffee chats, team channels                 |
| Tools         | Screen sharing, Loom videos, pair programming tools |
| Environment   | Ship equipment early, home office stipend           |

## Referencias

- Engineering Manager
- Cultura de Equipo
- Organización Típica
- Internal Developer Platform

# Arbol Deportes

## Team Topologies

Team Topologies es un framework de Matthew Skelton y Manuel Pais que define **4 tipos de equipo** y **3 modos de interacción** para diseñar organizaciones de software efectivas.

### Los 4 Tipos de Equipo

#### 1. Stream-Aligned Team

| Aspecto         | Detalle                                                   |
|-----------------|-----------------------------------------------------------|
| Propósito       | Alineado con un flujo de negocio (stream)                 |
| Responsabilidad | Build, run, y own un flujo de valor end-to-end            |
| Autonomía       | Alta — full ownership de su dominio                       |
| Ejemplos        | Equipo de checkout, equipo de onboarding, equipo de pagos |
| Tamaño          | 5-9 personas (two-pizza rule)                             |
| Skills          | Cross-functional: dev, QA, design, ops                    |

Características clave:

- Alineado con un product/service específico
- Minimiza handoffs y dependencias
- Puede hacer release independientemente
- Tiene ownership de la operación en producción

#### 2. Enabling Team

| Aspecto         | Detalle                                                          |
|-----------------|------------------------------------------------------------------|
| Propósito       | Ayudar a otros equipos a adquirir nuevas capacidades             |
| Responsabilidad | Coaching, tooling, best practices, knowledge transfer            |
| Autonomía       | Media — works across teams                                       |
| Ejemplos        | Equipo de DevOps adoption, Equipo de Testing, Equipo de Security |
| Tamaño          | 3-5 personas                                                     |
| Skills          | Deep expertise en un área específica                             |

Características clave:

- **Temporal:** existe hasta que los stream-aligned teams internalizan la capacidad
- **No owns code** de producción (en general)
- **Enables, no does:** enseña a pescar, no da el pez
- **Evolution:** una vez que el stream team es independiente, el enabling team se mueve a otro área

#### Ejemplo de enabling team en acción:

Mes 1-3: Enabling team de Kubernetes

- └ Week 1-2: Training para Team A
- └ Week 3-4: Pair programming con Team A
- └ Month 2: Team A deploya a K8s con soporte
- └ Month 3: Team A es independiente
- └ Month 4: Enabling team se mueve a Team B

### 3. Complicated-Subsystem Team

| Aspecto                | Detalle                                                        |
|------------------------|----------------------------------------------------------------|
| <b>Propósito</b>       | Manages un subsystem con alta complejidad técnica              |
| <b>Responsabilidad</b> | Build y maintain el subsystem                                  |
| <b>Autonomía</b>       | Media-alta dentro de su dominio                                |
| <b>Ejemplos</b>        | Motor de ML, video codec, motor de reglas fiscales, CAD engine |
| <b>Tamaño</b>          | 3-7 personas                                                   |
| <b>Skills</b>          | Deep domain expertise                                          |

#### Características clave:

- Alto nivel de expertise necesario (years to master)
- Interfaces bien definidas con el resto del sistema
- Puede tener su propio ciclo de release
- Requiere pessoas con background académico/ Research

### 4. Platform Team

| Aspecto                | Detalle                                                    |                               |
|------------------------|------------------------------------------------------------|-------------------------------|
| <b>Propósito</b>       | Proveer internal services para otros equipos               |                               |
| <b>Responsabilidad</b> | Build y operate la [../02-Estructura/14-plataforma-interna | Internal Developer Platform]] |
| <b>Autonomía</b>       | Alta sobre la plataforma                                   |                               |
| <b>Ejemplos</b>        | Platform engineering, DevOps platform, Data platform       |                               |
| <b>Tamaño</b>          | 4-8 personas                                               |                               |

| Aspecto | Detalle                                           |
|---------|---------------------------------------------------|
| Skills  | Infrastructure, developer experience, reliability |

#### Características clave:

- Treats other teams as **customers**
- Provides **self-service** capabilities
- Goal: reduce cognitive load de stream-aligned teams
- Enables autonomía de otros equipos

Ver Internal Developer Platform para más detalles.

## Los 3 Modos de Interacción

### 1. Collaboration

| Aspecto  | Detalle                                                 |
|----------|---------------------------------------------------------|
| Qué es   | Dos equipos trabajan juntos por un tiempo definido      |
| Cuándo   | Learning a new technology, solving a complex problem    |
| Duración | Temporal (semanas a meses)                              |
| Ejemplo  | Platform team + Stream team during Kubernetes migration |

### 2. X-as-a-Service

| Aspecto  | Detalle                                            |
|----------|----------------------------------------------------|
| Qué es   | Un equipo provee algo al otro con interfaz clara   |
| Cuándo   | Platform team provides tools, APIs, infrastructure |
| Duración | Permanente                                         |
| Ejemplo  | Platform team provides CI/CD as a service          |

### 3. Facilitating

| Aspecto  | Detalle                                                |
|----------|--------------------------------------------------------|
| Qué es   | Un equipo enseña y habilita al otro                    |
| Cuándo   | Enabling team helping stream teams adopt new practices |
| Duración | Temporal (hasta que el otro equipo es independiente)   |
| Ejemplo  | Security team helping all teams implement SAST         |

# Matriz de Topologías

| Tipo de Equipo        | Interacción Principal    | Interacción Secundaria             |
|-----------------------|--------------------------|------------------------------------|
| Stream-Aligned        | X-as-a-Service (consume) | Collaboration (con subsystem)      |
| Enabling              | Facilitating             | —                                  |
| Complicated-Subsystem | X-as-a-Service (provee)  | Collaboration (con stream)         |
| Platform              | X-as-a-Service (provee)  | Facilitating (enable stream teams) |

## Diseñando con Team Topologies

### Paso 1: Identificar los Streams de Valor

Business Value Streams:

- Customer Acquisition
  - Stream: Onboarding, Marketing, Sales
- Order Fulfillment
  - Stream: Checkout, Payment, Shipping
- Customer Support
  - Stream: Ticketing, Knowledge Base, Chat

### Paso 2: Asignar Stream-Aligned Teams

Stream Team: Checkout

- Owns: checkout flow, cart, payment initiation
- Interacts: Payment subsystem team (X-as-a-Service)
- Interacts: Platform team (CI/CD, infra)
- Size: 7 people

### Paso 3: Identificar Subsystems Complicados

Complicated Subsystem: Payment Processing Engine

- Owns: payment routing, fraud detection, compliance
- Provides: API for checkout team
- Size: 5 people (payment domain experts)

### Paso 4: Design Platform y Enabling Teams

Platform Team: Developer Platform

- Provides: CI/CD, monitoring, secrets, databases
- Interaction: X-as-a-Service con todos los teams
- Size: 6 people

Enabling Team: Security

- └─ Facilita: Security practices para todos los teams
- └─ Duration: 6 months, then rotates
- └─ Size: 3 people

## Team Topologies vs Spotify Model

| Aspecto                     | Team Topologies                     | Spotify Model                    |
|-----------------------------|-------------------------------------|----------------------------------|
| <b>Tipos de equipo</b>      | 4 types + interaction modes         | Squads, Tribes, Chapters, Guilds |
| <b>Focus</b>                | Organizational design for fast flow | Autonomy y alignment             |
| <b>Complexity</b>           | Simpler taxonomy                    | More roles, more structure       |
| <b>Interaction explicit</b> | Yes (3 modes)                       | Implicit                         |
| <b>Temporal teams</b>       | Yes (enabling teams)                | Not explicitly                   |
| <b>Conway alignment</b>     | Core principle                      | Emergent                         |

## Anti-patrones

| Anti-patrón                   | Problema                              | Solución                |
|-------------------------------|---------------------------------------|-------------------------|
| <b>No stream teams</b>        | Everything is a platform team         | Identify value streams  |
| <b>Permanent enabling</b>     | Enabling team never leaves            | Set exit criteria       |
| <b>Platform as gatekeeper</b> | Platform slows everyone down          | Self-service, API-first |
| <b>Subsystem sprawl</b>       | Too many complicated subsystem teams  | Merge or simplify       |
| <b>No interaction modes</b>   | Teams don't know how to work together | Define explicitly       |

## Referencias

- Squads, Tribes, Chapters
- Internal Developer Platform
- Equipos Autónomos
- Ley de Conway
- Microservicios

## Plataforma Interna

# Internal Developer Platform (IDP)

Una IDP es un layer de abstractions que permite a los desarrolladores provisionar y gestionar infraestructura de forma self-service, sin entender la complejidad subyacente.

## ¿Qué es una Internal Developer Platform?

Una IDP es el **punto único de interacción** entre los desarrolladores y toda la infraestructura, tooling, y servicios que necesitan para hacer su trabajo.

"An Internal Developer Platform is the sum of all the tech and tools that a platform team glues together to build golden paths that reduce cognitive load for developers." — Humanitec

## Componentes de una IDP

### Core Components

| Componente         | Qué hace                                      | Herramientas                      |
|--------------------|-----------------------------------------------|-----------------------------------|
| Developer Portal   | Interfaz unificada para todo                  | Backstage, Port, Cortex           |
| Service Catalog    | Registry de todos los servicios               | Backstage, Service Catalog        |
| Golden Paths       | Templates pre-aprobados para nuevos proyectos | Backstage Templates, Cookiecutter |
| CI/CD              | Build, test, deploy pipelines                 | GitHub Actions, GitLab CI, ArgoCD |
| Infrastructure     | Provisioning de cloud resources               | Terraform, Pulumi, Crossplane     |
| Secrets Management | Gestión de secrets y config                   | Vault, AWS Secrets Manager        |
| Observability      | Logs, metrics, traces unificados              | Grafana, Datadog, New Relic       |
| Service Mesh       | Networking entre servicios                    | Istio, Linkerd, Cilium            |

### Developer Experience Layer

| Capa            | Función                                     | Ejemplo                                  |
|-----------------|---------------------------------------------|------------------------------------------|
| CLI tools       | Comandos para interactuar con la plataforma | platform create-service, platform deploy |
| API             | Programmatic access a la plataforma         | REST/GraphQL API                         |
| UI              | Dashboard web                               | Backstage, custom portal                 |
| Git integration | Infra as code, PR-based workflows           | Terraform PRs, ArgoCD sync               |

# Backstage: El Estándar de la Industria

## ¿Qué es Backstage?

Backstage es un **developer portal** open source creado por Spotify y donado a la CNCF.

| Aspecto      | Detalle                                        |
|--------------|------------------------------------------------|
| Propósito    | Unificar tooling, servicios, y documentación   |
| Extensible   | Plugin system para customizar                  |
| Adoptado por | Spotify, Netflix, American Airlines, many more |
| CNCF Status  | Incubating project                             |

## Componentes de Backstage

```
Backstage
├── Software Catalog
│   ├── Services (APIs, websites, libraries)
│   ├── Resources (databases, S3 buckets)
│   ├── Components (microservices)
│   └── Systems (group of components)
├── Software Templates (Golden Paths)
│   ├── Create new service
│   ├── Create new library
│   ├── Create new documentation
│   └── Custom templates
├── TechDocs
│   ├── Documentation as code
│   ├── Auto-generated from markdown
│   └── Integrated con service catalog
├── Tech Radar
│   ├── Visualización de tecnología adopt/trial/assess/hold
│   └── Contributions from engineers
└── Plugins (100+)
    ├── Kubernetes
    ├── GitHub Actions
    ├── PagerDuty
    ├── Datadog
    └── Custom plugins
```

## Backstage vs Otras Soluciones

| Herramienta | Tipo               | Pricing            | Customizable |
|-------------|--------------------|--------------------|--------------|
| Backstage   | Open source portal | Free (self-hosted) | Highly       |

| Herramienta | Tipo                 | Pricing  | Customizable |
|-------------|----------------------|----------|--------------|
| Port        | SaaS portal          | Paid     | Moderately   |
| Cortex      | SaaS service catalog | Paid     | Moderately   |
| OpsLevel    | SaaS platform        | Paid     | Moderately   |
| Custom      | Build your own       | Dev cost | Fully        |

## Build vs Buy

### Build (Custom IDP)

| Aspecto     | Detalle                                                     |
|-------------|-------------------------------------------------------------|
| Ventajas    | Full control,定制 exact needs, no vendor lock-in              |
| Desventajas | High dev cost (2-4 engineers full-time), maintenance burden |
| Cuándo      | Org con requirements muy específicos, budget alto           |
| Timeline    | 6-12 months para MVP                                        |

### Buy (SaaS Solutions)

| Aspecto     | Detalle                                      |
|-------------|----------------------------------------------|
| Ventajas    | Fast setup, maintained by vendor, support    |
| Desventajas | Less custom, ongoing cost, vendor dependency |
| Cuándo      | Org want fast time-to-value, standard needs  |
| Timeline    | 2-4 weeks para setup                         |

### Hybrid (Backstage + Custom Plugins)

| Aspecto     | Detalle                                                     |
|-------------|-------------------------------------------------------------|
| Ventajas    | Open source base, customizable, large community             |
| Desventajas | Still need dev effort for plugins, Backstage learning curve |
| Cuándo      | Most organizations — the sweet spot                         |
| Timeline    | 2-3 months para useful MVP                                  |

## Framework de Decisión

```
¿Tienes un equipo de platform dedicado?
├─ Sí (3+ engineers)
│   ├── ¿Requirements estándar? → Backstage + plugins
│   └── ¿Requirements únicos? → Custom IDP
├─ No
│   ├── ¿Budget para SaaS? → Port, Cortex, OpsLevel
│   └── ¿Budget limitado? → Backstage + managed hosting
└─ ¿Quieres empezar rápido?
    ├── Sí → SaaS o Backstage con templates básicos
    └─ No → Custom IDP
```

## Golden Paths

Los Golden Paths son **templates pre-aprobados** que guían a los desarrolladores por el camino correcto.

### Ejemplo: Golden Path para Nuevo Microservicio

```
# backstage/template.yaml
apiVersion: scaffolder.backstage.io/v1beta3
kind: Template
metadata:
  name: microservice-template
  description: Create a new microservice with best practices
  tags:
    - microservice
    - java
    - spring-boot
spec:
  owner: platform-team
  type: service
  steps:
    - id: fetch-template
      action: fetch:template
      input:
        url: ./template
    - id: create-repo
      action: github:repo:create
      input:
        name: ${ parameters.name }
    - id: setup-cicd
      action: github:actions:workflow
      input:
        workflowPath: .github/workflows/ci.yml
    - id: register-catalog
      action: catalog:register
      input:
        catalogInfoPath: /catalog-info.yaml
```

**Lo que el developer obtiene:**

- Repo con estructura estándar
- CI/CD pipeline configurado
- Service catalog entry
- Observability hooks
- Documentation template
- Security scanning integrado

## Métricas de una IDP

| Métrica                            | Target                    | Cadencia    |
|------------------------------------|---------------------------|-------------|
| Developer satisfaction (DX score)  | > 4/5                     | Quarterly   |
| Time to first deploy (new service) | < 1 día                   | Per service |
| Self-service ratio                 | > 80%                     | Monthly     |
| Platform adoption rate             | > 90% del equipo          | Monthly     |
| Golden path usage                  | > 70% de nuevos proyectos | Monthly     |
| Time to onboard (new engineer)     | < 5 días                  | Per hire    |

## Referencias

- Team Topologies
- Plataformas de Ingeniería
- Onboarding
- Equipos Autónomos
- DevOps e Infra
- Herramientas

# README

## Estructura

Organización, roles, gobierno técnico y modelos de equipo que dan forma a una Software Factory moderna.

### Contenido

| #  | Archivo                                    | Tema                            | Nivel                                                 |              |
|----|--------------------------------------------|---------------------------------|-------------------------------------------------------|--------------|
| 01 | [[../02-Estructura/01-organizacion-tipica  | Organización Típica]]           | Jerarquía, departamentos, líneas de reporte           | Principiante |
| 02 | [[../02-Estructura/02-roles-equipo         | Roles del Equipo]]              | Dev, QA, DevOps, Architect, PM, PO, SM, Designer, SRE | Principiante |
| 03 | [[../02-Estructura/03-tech-lead            | Tech Lead]]                     | Decisiones técnicas, mentoring, code review           | Intermedio   |
| 04 | [[../02-Estructura/04-architect            | Software Architect]]            | Arquitectura, ADRs, tech radar                        | Avanzado     |
| 05 | [[../02-Estructura/05-engineering-manager  | Engineering Manager]]           | People management, 1:1s, career growth, hiring        | Intermedio   |
| 06 | [[../02-Estructura/06-product-owner        | Product Owner]]                 | Backlog, priorización, user stories                   | Intermedio   |
| 07 | [[../02-Estructura/07-squad-tribe-chapter  | Squads, Tribes, Chapters]]      | Modelo Spotify, pros/contras                          | Intermedio   |
| 08 | [[../02-Estructura/08-gobernanza-tecnica   | Gobernanza Técnica]]            | Tech Council, RFCs, design reviews                    | Avanzado     |
| 09 | [[../02-Estructura/09-adr-decisiones       | Architecture Decision Records]] | Formato ADR, herramientas, ejemplos                   | Intermedio   |
| 10 | [[../02-Estructura/10-equipos-autonomos    | Equipos Autónomos]]             | Autonomía, alineación, guardrails                     | Intermedio   |
| 11 | [[../02-Estructura/11-estructura-matricial | Estructura Matricial]]          | Patrones organizacionales                             | Intermedio   |
| 12 | [[../02-Estructura/12-onboarding-equipo    | Onboarding]]                    | Plan 30-60-90, buddy system                           | Principiante |
| 13 | [[../02-Estructura/13-arbol-deportes       | Team Topologies]]               | 4 tipos de equipo, modos de interacción               | Avanzado     |
| 14 | [[../02-Estructura/14-plataforma-interna   | Internal Developer Platform]]   | IDP, Backstage, build vs buy                          | Avanzado     |

## Secciones relacionadas

- [01 — Fundamentos](#)
- [03 — Procesos](#)
- [04 — Prácticas](#)
- [05 — Herramientas](#)
- [07 — IA y Software](#)